Chapter 7
Relational Algebra & Calculus

- Algebra
  - **Defines a set of operations** for the relational model
  - The operations enable a user to specify basic **retrieval requests**.
  - The algebra operations **produce new relations**, which can be further manipulated.
  - A sequence of algebra operations forms a **relational algebra expression.**

Select sname, city

From S

Where sno = ‘s1’;

S

SNAME	CITY
SAMAN	COLOMBO

S

SNO	SNAME	STAT
S1	SAMAN	
S2	JAGATH	
S3	ANIL	
S4	GAMINI	
S5	BALA	

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- Algebra is important
  - It provides **a formal foundation** for relational model operations
  - It is used as a basis for implementing and optimizing queries in RDBMS.

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- Algebra operations can divided into two groups
  - Set operations
    - Union, intersection, difference, Cartesian product
    - These are binary operations
  - Special operations
    - Select, project, join
    - Select and project are unary operations

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- Binary operations
- Two relations $R(A_1, A_2, \ldots A_n)$ and $S(B_1, B_2, \ldots, B_n)$ are said to be **union compatible** if they have the same degree n and if $dom(A_i) = dom(B_i)$ for $1 <= i <= n$.
- Union – The result of this operation is denoted by $R \cup S$, is a relation that includes all tuples that are either in R or in S or in both R and S. Duplicate tuples are eliminated.
- R(sno, sname, city)
- S(sno, sname, city)

SNO
S1
S2
S3
S4
S5
S6

SNO
S1
S2
S3
S4
S5

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- Intersection – The result of this operation is denoted by $R \cap S$, is a relation that includes all tuples that are in both R and S.
- Difference (Minus) – The result of this operation is denoted by R-S, is a relation that includes all tuples that are in R but not in S.
- R and S should be union compatible
- Union and intersection are commutative and associative

  $R \cup S = S \cup R$ and $R \cap S = S \cap R$

  $R \cup (S \cup T) = (R \cup S) \cup T$ and

  $(R \cap S) \cap T = R \cap (S \cap T)$
- But minus is not commutative

  $R - S \neq S - R$

SNO
S1
S2
S3
S4
S5
S6

SNO
S1
S2
S3
S4
S5

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- Cartesian product

The cartesian product of two relations $R(A_1, A_2, \ldots A_n)$ X $S(B_1, B_2, \ldots, B_m)$ is a relation Q with degree **n+m attributes** $Q(A_1, A_2, \ldots A_n, B_1, B_2, \ldots, B_m)$ in that order.

The resulting relation has Q has one tuple for each combination of tuples. If R has n_r tuples and S has n_s tuples then Q has

n_r * n_s tuples.

SB1
t1b1
t2b1

R(A1
S(B1
Q(A1

R: nr
S: ns
Q: nc

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- Unary operations
- Select
  - Let R be a relation and C be a specified condition, where C is a Boolean expression based on the relation R comprising
    - Constants and/or attributes of the given relation
    - Logical operations – and/ or/ not
    - Comparison operations

Then the selection of R under the specified condition C is the set of all tuples t satisfying the condition C

This is denoted by

$$\sigma_{<\text{selection condition}>}R$$

eg; $\sigma_{<\text{city} = \text{'Colombo'}>}$Supplier

$\sigma_{<\text{sno}= \text{'s1'}>}S$

Select sname, city
From **S**
Where **sno = 's1';**

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- The degree of the relation resulting from the select is same as the degree of R.
- Select operation is commutative
  - $\sigma_{<cond1>}(\sigma_{<cond2>}(R)) = \sigma_{<cond2>}(\sigma_{<cond1>}(R))$
- Can combine a cascade of select into a single select operation
  - $\sigma_{<cond1>}(\sigma_{<cond2>}(\ldots\ldots \sigma_{<condn>}(R)) = \sigma_{<cond1>and<cond2>and\ldots\ldots and<condn>}(R)$

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- Project

Let R be a relation. Then the project or R on attributes X, Y,…Z is the set of all tuples (x, y,….z)such that a tuple t appears in R with X – value x, Y – value y, ……. and Z – value z.

Denoted by

$\pi_{<attribute\ list>}(R)$

eg; $\pi_{sex,\ salary}(Employee)$

$\pi_{sname,\ city}(\sigma_{<sno=\ 's1'>}S)$

Select sname,city
From S
Where sno = 's1';

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- The resultant relation has only the attributes specified in the attribute list. $\pi_{sname,\ city}(\sigma_{<sno=\ 's1'>}S)$
- Therefore the degree is equal to the number of attributes in the attribute list.
- The project operation removes any duplicate tuples.

$\pi_{<list1>}(\pi_{<list2>}(R)) = \pi_{<list1>}(R)$

as long as <list2> contains the attributes in <list1> $\pi_{sname}(\pi_{sname,\ city}(S)) = \pi_{sname}(S)$

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- Join operation
- Denoted by $\bowtie$
- Used to combine related tuples from two relations into single tuples.
- Let θ be any valid comparison operation. Let R and S be two relations.

Then $R \bowtie_{R.X\ \theta\ S.Y} S$ is the set of all tuples *t* s.t. *t* is the concatenation of a tuple *r* belonging to R and a tuple *s* belonging to S and the condition R.X θ S.Y evaluates to true.

The attributes X and Y should be in the same domain and the operation θ should make sense for that domain.

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- If θ is = it is called equijoin.
- The result of an equijoin always have one or more pairs of attributes that have identical values in every tuple.
- Natural join denoted by * get rids of the second attribute in an equijoin condition.
- Natural join requires that the two join attributes have the same name in both relation.

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- Renaming attributes

Attributes can be renamed in the intermediate and result relation.

Eg:

$Temp \leftarrow \sigma_{DNO = 5}$ (Employee)

$R(Firstname, Lastname, Salary) \leftarrow \pi_{Fname, Lname, Salary}$ (Temp)

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- Renaming relations

ρ - is used to rename relations

Consider the relation schema $R(A_1, A_2, \ldots A_n)$

$\rho_{S(B1, B2,\ldots,Bn)}(R)$ – renames relation and attributes

$\rho_{S}(R)$ – renames only relation

$\rho_{(B1, B2,\ldots,Bn)}(R)$ – renames only attributes

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- $\{\sigma, \pi, \cup, -, x\}$ is known as a complete set.
- Other operations can be expressed as a sequence of operations from this set.

$$R \cap S \equiv (R \cup S) - ((R-S) \cup (S-R))$$

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- Division operation
- A special operation used for special kind of queries.

Division is applied to two relations R(Z) ÷ S(X), where $X \subseteq Z$. Let Y = Z – X. The result of the division is a relation T(Y) that includes a tuple *t* if tuple t_R appear in R with $t_R[Y] = t$ and with $t_R[X] = t_S$ for every tuple t_S in S.

This means a tuple t to appear in the result, the values in t must appear in R in combination with every tuple in S.

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- Aggregate functions and grouping
- Written as
- <grouping attributes> $\mathcal{F}$ <function list> (R)
- <grouping attributes> is a list of attributes of the relation in R
- <function list> is a (<function> <attribute>) pair. In each pair <function> is one of the aggregate function . (eg. Sum, average, etc)
- The resulting relation has the grouping attributes plus one attribute for each element in the function list.

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

$_{dno}\Gamma_{\ count\ SSN\ AVERAGE\ salary}$ (Employee)

$\rho\ R_{(dno,\ no_emp, avg_sal)}\ {}_{dno}\Gamma_{\ count\ SSN\ AVERAGE\ salary}$ (Employee)

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- Outer join
- In natural join, tuples without a matching tuple are eliminated from the join result.
- Tuples with null values in the join attributes are also eliminated.
- This amounts to loss of information, if the result is necessary to be based on all the information in the component relation.

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- Left outer join

Denoted by ⟕

R ⟕ S keeps every tuple in R; if no matching tuple is found in S, the attributes of S in the join result are filled with null values.

Temp ← Employee $\bowtie_{ssn=mgrssn}$ (Department)

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- Right outer join

R ⟖ S keeps every relation in the second of right relation S

- Full outer join

R ⟗ S keeps all tuples in both left and right relations.

When no matching tuples are found null values are padded.

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

Relational Calculus

- In RC one declarative expression is written to specify a retrieval request. There is no description of how to evaluate a query. I.e. specifies what is to be retrieved rather than how to retrieve it.
- In RA we write a sequence of operations. Specify the strategy for evaluating the query.
- RC is declarative or non-procedural
- RA is procedural

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- Algebra and calculus are equivalent to one and other.
- A relational query language L is called **relationally complete** if we can express in L any query that can be expressed in RC.
- Two kinds of RC
  - Tuple oriented RC
  - Domain oriented RC

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- A general expression of the TC is of the form

$\{t_1.A_1, t_2.A_2, \ldots\ldots, t_n.A_n \mid Cond(t_1, t_2, \ldots\ldots t_n, t_{n+1}, \ldots t_{n+m})\}$

$t_1, t_2, \ldots\ldots t_n, t_{n+1}, \ldots t_{n+m}$ – tuple variables

Each A_i – an attribute of the relation on which t_i ranges

Cond – Well formed formula (WFF)

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- Tuple relational calculus
- Based on tuple variables and well formed formula
- A tuple variable t of a relation R is a variable whose range is the set of all tuples in R.

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- A WFF (Well-Formed Formula) is a combination of one or more of the following components.
  - Elementary condition
    - Boolean expressions involving two values either constants or attribute values and one of the comparison operators
  - Boolean operators
  - Universal quantifier (∀)
  - Existential quantifier (∃)

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

- If F1 and F2 are formulas then are so
  - F1 and F2
  - F1 of F2
  - Not F1
  - (∀ t)(F1) –true if F evaluates to true for some tuple assigned to free occurrences of t in F
  - (∃ t)(F1)

    t is a tuple variable

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

Free and bound variables

- Each occurrence of a tuple variable in a WFF is either free or bound.
- A tuple variable t occurs within a given WFF either as a attribute reference or as the variable immediately following one of the quantifiers ∀ or ∃
- A tuple variable t is bound if it appears in an (∀t) or (∃t) clauses, otherwise it is free.

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

Eg:

d.dname = “research”

(∃ t)(d.dnumber = t.dno)

d is free in both F1 and F2

t is bound in F2

Copyright © 2004 Ramez Elmasri and Shamkant Navathe

Tuple calculus expression

- Let P, Q,….,R be relations. Let P_1, ….P_m, Q_n,….Q_p,……,R_q,…..,R_r be attributes belonging to P,Q and R respectively. Then the expression

$\{P_x.P_1,\ldots..,P_x.P_m,Q_x.Q_n,\ldots\ldots Q_x.Q_p,\ldots\ldots,R_x.R_q,\ldots..,R_x.R_x \mid Cond(F)\}$

is called a tuple calculus where P_x ,Q_x , R_x are tuple variables of P,Q,…,R and F is a WFF containing exactly P_x ,Q_x , R_x as free variables.

Copyright © 2004 Ramez Elmasri and Shamkant Navathe